Practical 2 MI Lab

Name : Ishan Manekar

Class : A_A3_45

In [1]:
```
import pandas as pd #data science
import numpy as np #calculations
#visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:
```
# loading data set

df = pd.read_csv("USA_Housing.csv")

df.head()
```

Out[2]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Add
0	79545.458574	5.682861	7.009188	4.09	23086.800503	1.059034e+06	208 Michael Ferry Apt. 674\nLaura NE 3
1	79248.642455	6.002900	6.730821	3.09	40173.072174	1.505891e+06	188 Johnson Views Suite 079\r Kathleen,
2	61287.067179	5.865890	8.512727	5.13	36882.159400	1.058988e+06	9127 Elizabeth Stravenue\nDaniel WI 06
3	63345.240046	7.188236	5.586729	3.26	34310.242831	1.260617e+06	USS Barnett\nFPO AP 4
4	59982.197226	5.040555	7.839388	4.23	26354.109472	6.309435e+05	USNS Raymond\nFPO AE 0

In [3]:
```
df.tail()
```

Out[3]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price	Adc
4995	60567.944140	7.830362	6.137356	3.46	22837.361035	1.060194e+06	USNS Williams\nFPO AP 3(
4996	78491.275435	6.999135	6.576763	4.02	25616.115489	1.482618e+06	PSC 9258, Box 8489\nAP 42991-
4997	63390.686886	7.250591	4.805081	2.13	33266.145490	1.030730e+06	4215 Tracy Garden 076\nJoshualand, VA
4998	68001.331235	5.534388	7.130144	5.44	42625.620156	1.198657e+06	USS Wallace\nFPO AE 7
4999	65510.581804	5.992305	6.792336	4.07	46501.283803	1.298950e+06	37778 George Ridge 509\nEast Holly, N

In [4]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                       Non-Null Count  Dtype  
---  ------                       --------------  -----  
 0   Avg. Area Income             5000 non-null   float64
 1   Avg. Area House Age          5000 non-null   float64
 2   Avg. Area Number of Rooms    5000 non-null   float64
 3   Avg. Area Number of Bedrooms 5000 non-null   float64
 4   Area Population              5000 non-null   float64
 5   Price                        5000 non-null   float64
 6   Address                      5000 non-null   object 
dtypes: float64(6), object(1)
memory usage: 273.6+ KB
```

In [5]:
```
df.describe()
```

Out[5]:

	Avg. Area Income	Avg. Area House Age	Avg. Area Number of Rooms	Avg. Area Number of Bedrooms	Area Population	Price
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	5.000000e+03
mean	68583.108984	5.977222	6.987792	3.981330	36163.516039	1.232073e+06
std	10657.991214	0.991456	1.005833	1.234137	9925.650114	3.531176e+05
min	17796.631190	2.644304	3.236194	2.000000	172.610686	1.593866e+04
25%	61480.562388	5.322283	6.299250	3.140000	29403.928702	9.975771e+05
50%	68804.286404	5.970429	7.002902	4.050000	36199.406689	1.232669e+06
75%	75783.338666	6.650808	7.665871	4.490000	42861.290769	1.471210e+06
max	107701.748378	9.519088	10.759588	6.500000	69621.713378	2.469066e+06

In [6]:
```
df.isnull().sum()
```

Out[6]:
```
Avg. Area Income              0
Avg. Area House Age           0
Avg. Area Number of Rooms     0
Avg. Area Number of Bedrooms  0
Area Population               0
Price                         0
Address                       0
dtype: int64
```

In [7]:
```
df.drop("Address" , axis=1, inplace =True) # droping the address feature because no numeric value available
```

In [8]:
```
df.isnull().sum()
```

Out[8]:
```
Avg. Area Income                0
Avg. Area House Age             0
Avg. Area Number of Rooms       0
Avg. Area Number of Bedrooms    0
Area Population                 0
Price                           0
dtype: int64
```

In [9]:
```
df.duplicated().sum()
```

Out[9]:
```
0
```

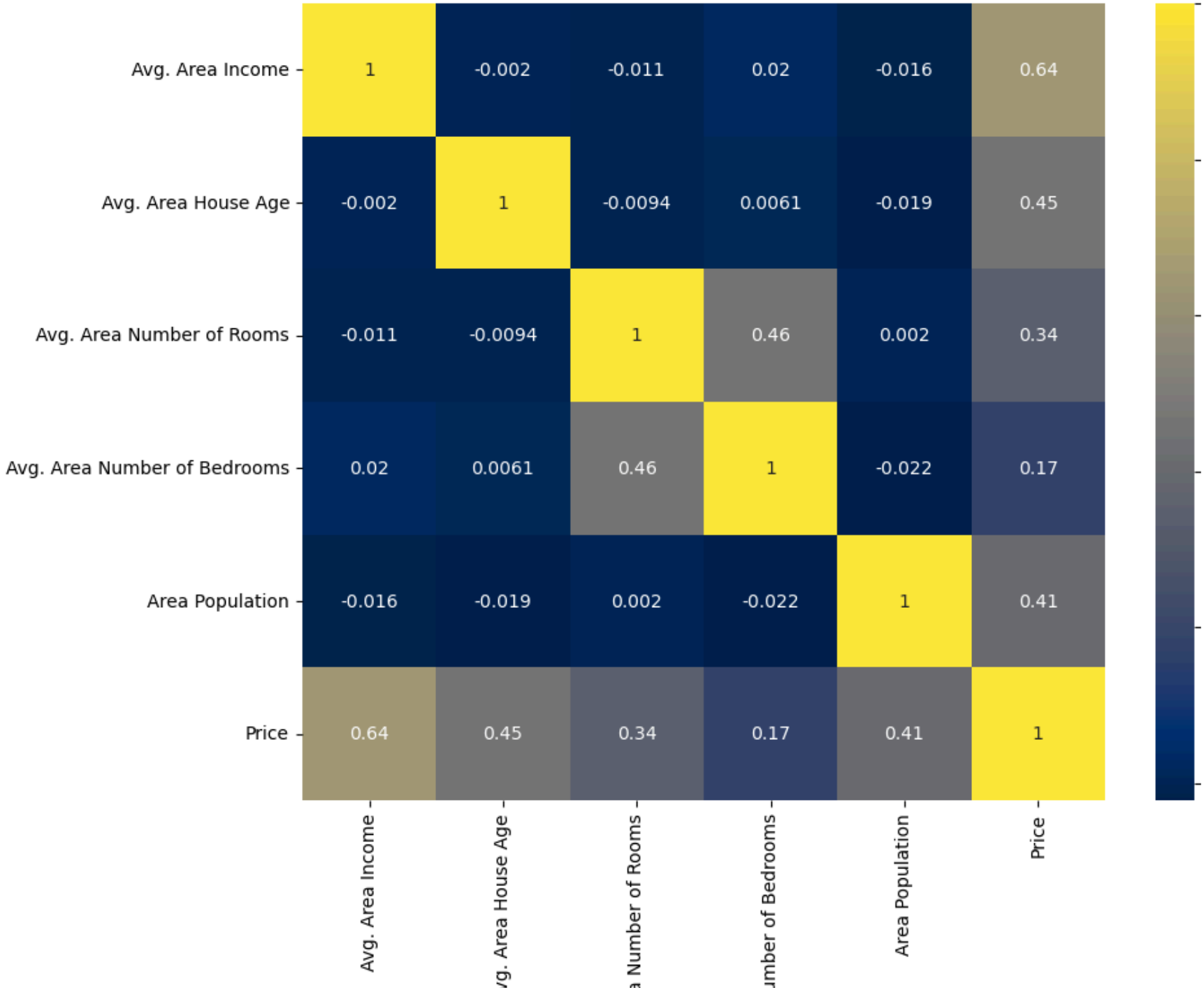
In [10]:
```
plt.figure(figsize=(10,8))

numeric_df = df.select_dtypes(include = ['number'])

sns.heatmap(numeric_df.corr(),
            annot =True,
            cmap='cividis')

plt.show()

#virdis, plasma, inferno, magma, cividis ye sab colours hai seaborn ke
```

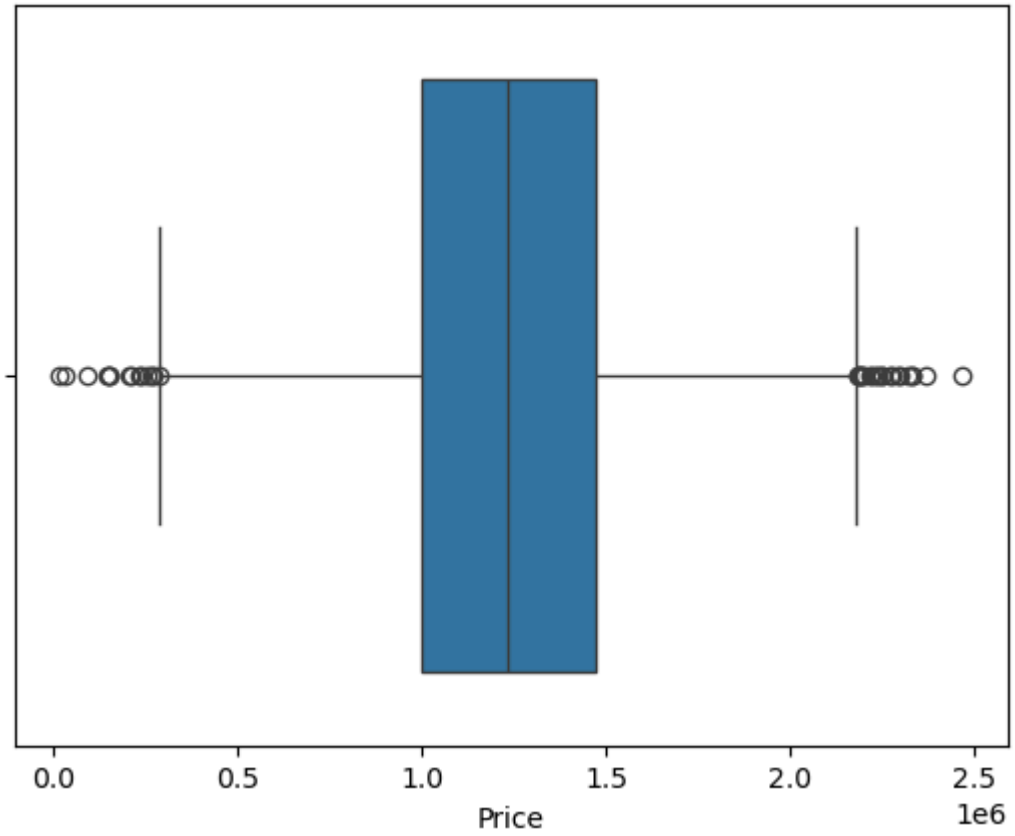

Avg. Area Income
Avg. Area House Age
Avg. Area Number of Rooms
Avg. Area Number of Bedrooms
Area Population
Price
1 -0.002 -0.011 0.02 -0.016 0.64
-0.002 1 -0.0094 0.0061 -0.019 0.45
-0.011 -0.0094 1 0.46 0.002 0.34
0.02 0.0061 0.46 1 -0.022 0.17
-0.016 -0.019 0.002 -0.022 1 0.41
0.64 0.45 0.34 0.17 0.41 1
Avg. Area Income
Area Population
Price

Avg. Area

Avg. Area N

In [ ]:

In [11]:
```
#boxplot of price
sns.boxplot(x=df["Price"])
plt.show()
```



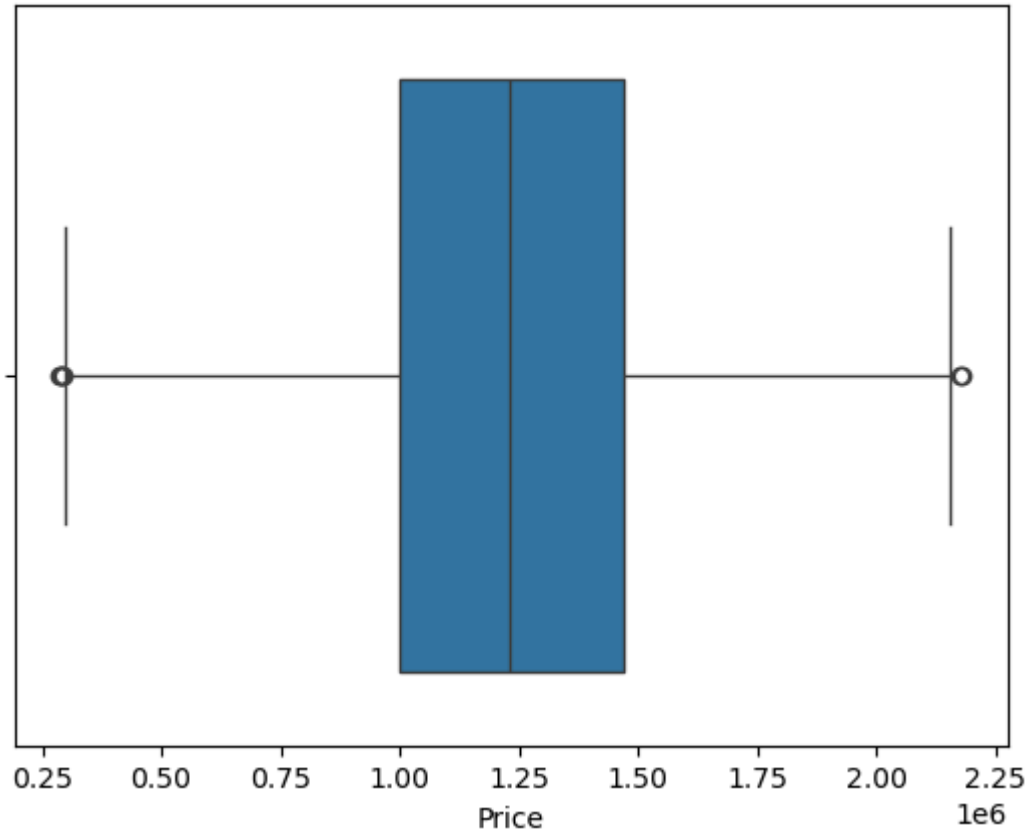
In [12]:
```
#outliers removing
Q1 = df["Price"].quantile(0.25)
Q3 = df["Price"].quantile(0.75)

IQR = Q3-Q1

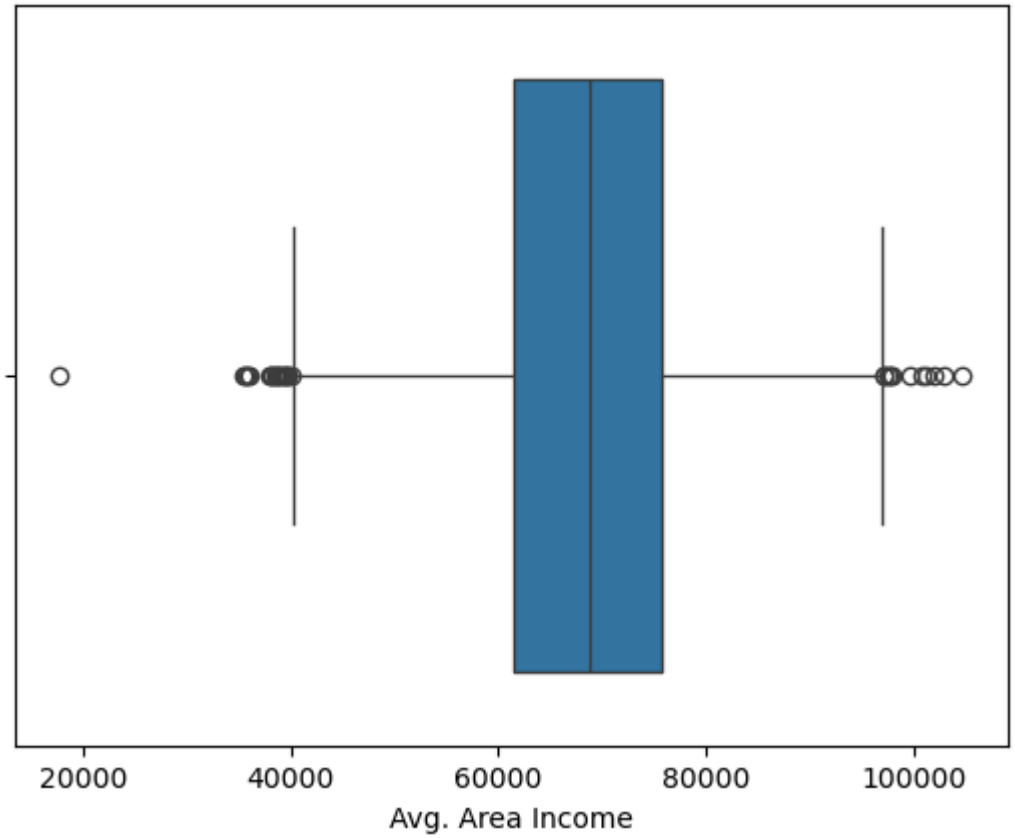
lower = Q1 -1.5*IQR
higher =Q3+ 1.5*IQR

df= df[(df["Price"] >= lower) & (df["Price"] <= higher)]
```

In [13]:
```
#boxplot of price after removing outliers
sns.boxplot(x=df["Price"])
plt.show()
```



In [14]:
```
#boxplot for avg area income
sns.boxplot(x=df["Avg. Area Income"])
plt.show()
```



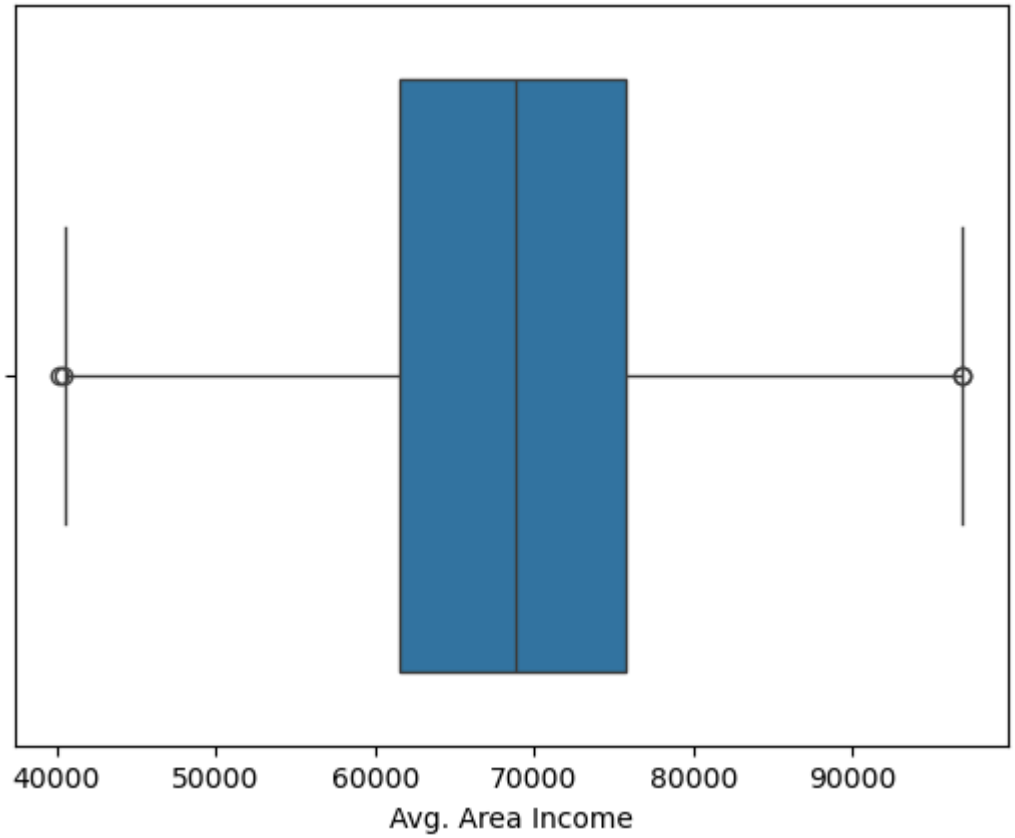
In [15]:
```
#outliers removing
Q1 = df["Avg. Area Income"].quantile(0.25)
Q3 = df["Avg. Area Income"].quantile(0.75)

IQR = Q3-Q1

lower = Q1 -1.5*IQR
higher =Q3+ 1.5*IQR

df= df[(df["Avg. Area Income"] >= lower) & (df["Avg. Area Income"] <= higher)]
```

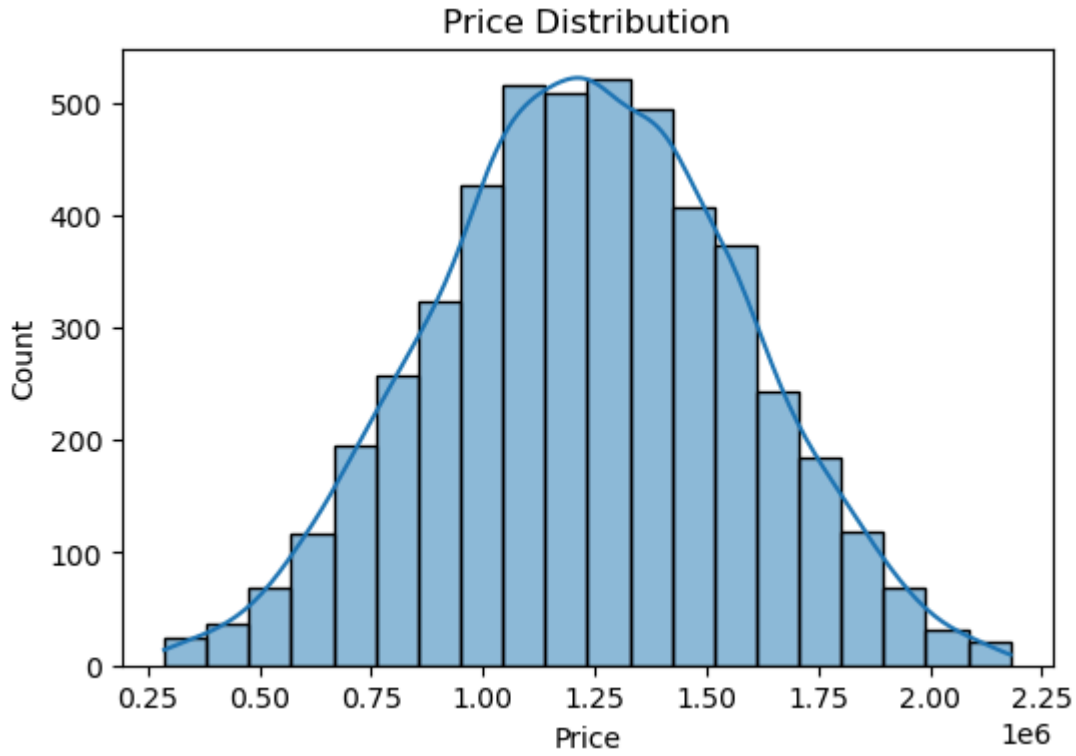
In [16]:
```
#boxplot for avg area income after removing outliers
sns.boxplot(x=df["Avg. Area Income"])
plt.show()
```



In [17]:
```
#Univariate Analysis Fare

plt.figure(figsize=(6,4))
sns.histplot(df["Price"], bins=20, kde=True)

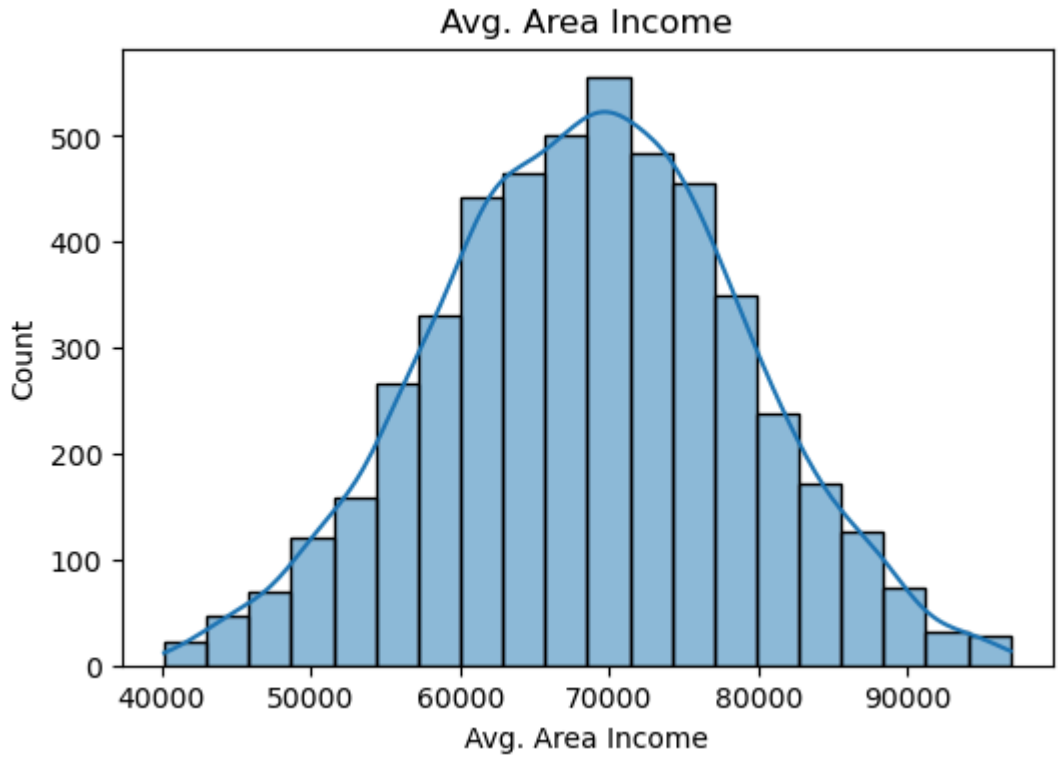
plt.title("Price Distribution")
plt.show()
```



In [18]:
```
#Univariate Analysis Fare

plt.figure(figsize=(6,4))
sns.histplot(df["Avg. Area Income"], bins=20, kde=True)

plt.title("Avg. Area Income")
plt.show()
```



SLR

In [19]:
```
X = df[["Avg. Area Income"]]
y = df ["Price"]
```

In [20]:
```
#train test split

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X,y, test_size =0.30, random_state = 42)
```

In [21]:
```
from sklearn.linear_model import LinearRegression

slr_model = LinearRegression()

slr_model.fit(X_train,y_train)
```

Out[21]:
```
LinearRegression()
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [22]:
```
y_pred = slr_model.predict(X_test)
```

In [23]:
```
from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import mean_absolute_error
from sklearn.metrics import r2_score

import numpy as np
print("Simple Linear Regression Results")
print("---------------------------------")

print("MAE :", mean_absolute_error(y_test, y_pred))
print("MSE :",mean_squared_error(y_test,y_pred))
print("RMSE :", np.sqrt(mean_squared_error(y_test,y_pred)))
print("R2 score: ", r2_score(y_test,y_pred))
```

```
Simple Linear Regression Results
---------------------------------
MAE : 207525.98140090736
MSE : 65966799949.412445
RMSE : 256840.02793453448
R2 score:  0.4252225135683304
```

In [24]:
```
print("Intercept: ", slr_model.intercept_)
print("Slope: ", slr_model.coef_[0])
```

```
Intercept:  -158790.85730962176
Slope:  20.296835619768096
```

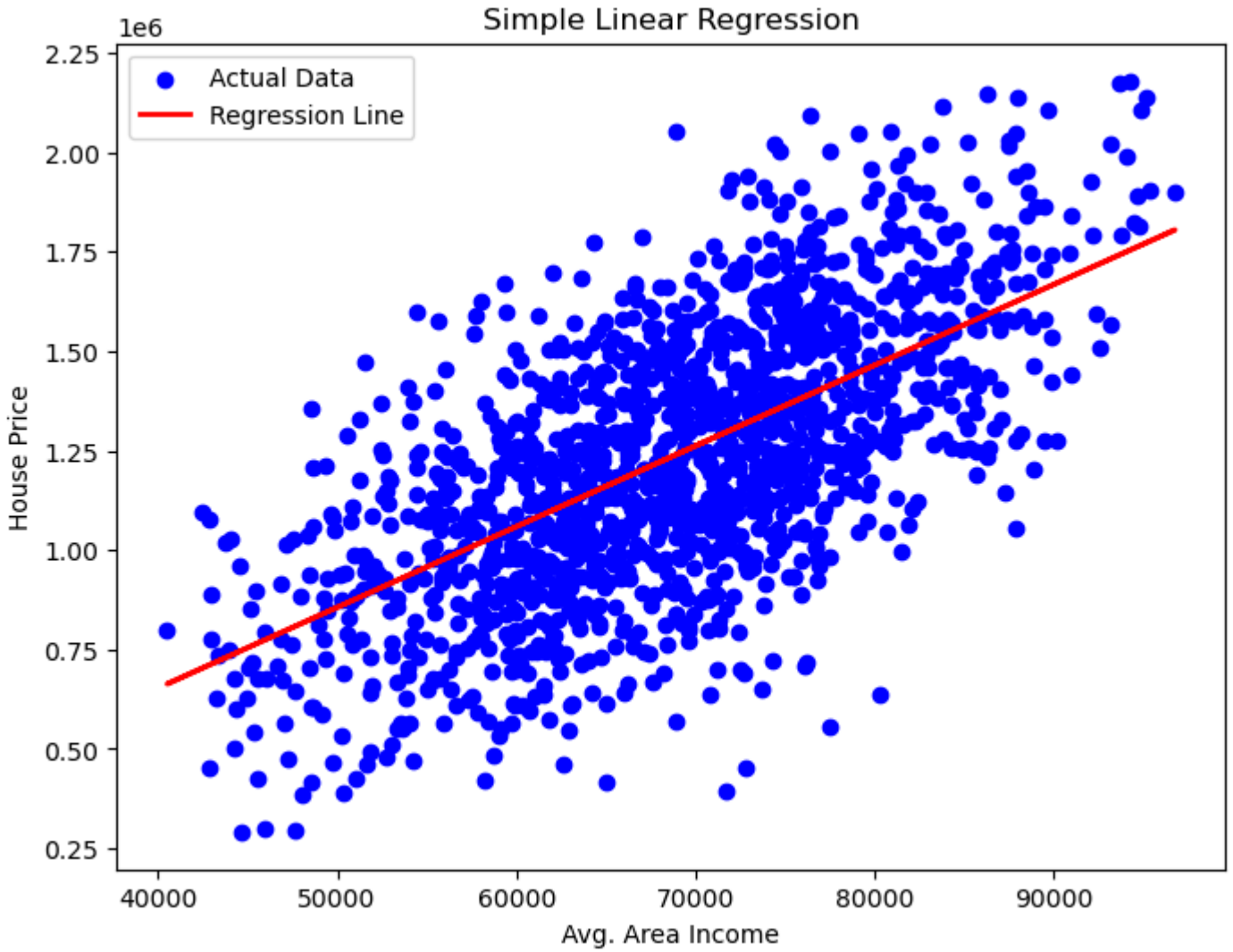
In [28]:
```
plt.figure(figsize=(8,6))

plt.scatter(X_test,y_test,color='blue',label ='Actual Data')

plt.plot(X_test,y_pred,color='red', linewidth=2,label ='Regression Line')

plt.xlabel("Avg. Area Income")
plt.ylabel("House Price")
plt.title("Simple Linear Regression")
plt.legend()

plt.show()
```


Simple Linear Regression
1e6
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
House Price
Actual Data
Regression Line
40000
50000
60000
70000
80000
90000
Avg. Area Income

In [26]:
```
try:
    # 1. Take input and safely convert to float
    income = float(input("Enter Avg. Area Income: "))

    # 2. Reshape into the 2D array format required by scikit-learn
    new_data = np.array([[income]])

    # 3. Generate the prediction
    prediction = slr_model.predict(new_data)

    # 4. Extract the scalar value and print formatted currency
    # Using .item() or [0][0] prevents errors if the model returns a 2D array
    predicted_price = prediction.item()

    print("Predicted House Price = ${:,.2f}".format(predicted_price))

except ValueError:
    print("Error: Please enter a valid numerical value for income.")
except NameError:
    print("Error: Make sure 'slr_model' is trained and 'np' is imported.")
```

```
Enter Avg. Area Income:  23457

Predicted House Price = $317,312.02

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names,
t LinearRegression was fitted with feature names
  warnings.warn(
```

Type *Markdown* and LaTeX: α^2

Multiple linear regression

In [31]:
```
X = df.drop(["Price"], axis=1)

y = df["Price"]
```

In [33]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size = 0.2,
    random_state =42)
```

In [34]:
```
mlr = LinearRegression()

mlr.fit(X_train, y_train)
```

Out[34]:
```
LinearRegression()
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [35]:
```
coef = pd.DataFrame(
    mlr.coef_,
    X.columns,
    columns=["Coefficient"])

coef
```

Out[35]:

	Coefficient
Avg. Area Income	21.505060
Avg. Area House Age	166528.530395
Avg. Area Number of Rooms	120171.483174
Avg. Area Number of Bedrooms	1715.111080
Area Population	15.037670

In [36]:
```
y_pred =mlr.predict(X_test)
```

In [38]:
```
print("MAE :", mean_absolute_error(y_test,y_pred))

print("MSE :", mean_squared_error(y_test,y_pred))

print("RMSE :", np.sqrt(mean_squared_error(y_test,y_pred)))

print("R2 Score :", r2_score(y_test,y_pred))
```

```
MAE : 79279.44669409566
MSE : 9993387098.255205
RMSE : 99966.93002315919
R2 Score : 0.9148267001804845
```

In [39]:
```
plt.figure(figsize=(6,6))

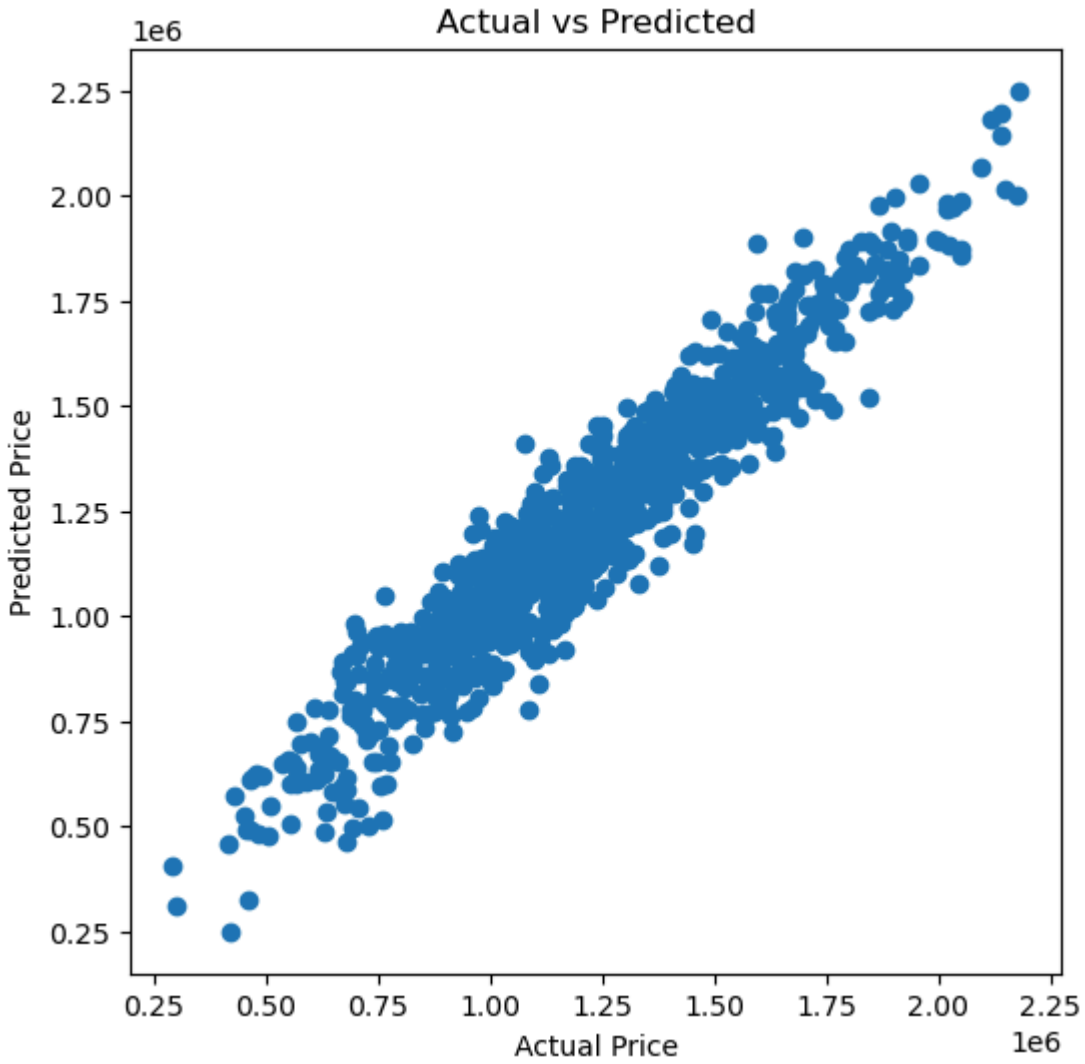
plt.scatter(y_test,y_pred)

plt.xlabel("Actual Price")

plt.ylabel("Predicted Price")

plt.title("Actual vs Predicted")

plt.show()
```


Actual vs Predicted
1e6
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
Predicted Price
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
2.25
1e6
Actual Price

In [41]:
```
#take input from the user
income = float(input("Enter the avg area income:"))
house_age = float(input("Enter avg area house age:"))
rooms = float(input("Enter the avg area number of rooms:"))
bedrooms = float(input("Enter the avg area number of bedrooms:"))
population = float(input("Enter the area population:"))
test_input=np.array([[income, house_age,rooms,bedrooms,population]])

predicted_price = mlr.predict(test_input)

print("\n Predicted House Price: ${:,.2f}".format(predicted_price[0]))
```

```
Enter the avg area income: 40000
Enter avg area house age: 15
Enter the avg area number of rooms: 7
Enter the avg area number of bedrooms: 4
Enter the area population: 100000


 Predicted House Price: $3,081,510.69

C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:493: UserWarning: X does not have valid feature names,
t LinearRegression was fitted with feature names
  warnings.warn(
```

Ridge Regression

In [42]:
```
from sklearn.model_selection import GridSearchCV
```

In [43]:
```
from sklearn.linear_model import Ridge

ridge = Ridge()

ridge.fit(X_train,y_train)

ridge_pred = ridge.predict(X_test)

print("Default Ridge R2 :",r2_score(y_test,ridge_pred))
```

```
Default Ridge R2 : 0.914832076303878
```

In [44]:
```
param_grid={'alpha': [0.001,0.01,0.1,1,10,100]}
```

In [45]:
```
grid_ridge = GridSearchCV(estimator = Ridge(),param_grid=param_grid,
                          scoring='r2',
                          cv=5)
```

In [46]:
```
grid_ridge.fit(X_train,y_train)
```

Out[46]:
```
GridSearchCV(cv=5, estimator=Ridge(),
             param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10, 100]},
             scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [47]:
```
grid_ridge.fit(X_train,y_train)
```

Out[47]:
```
GridSearchCV(cv=5, estimator=Ridge(),
             param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10, 100]},
             scoring='r2')
```

In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [48]:
```
print("Best Alpha :", grid_ridge.best_params_)
```

```
Best Alpha : {'alpha': 1}
```

In [49]:
```
print(grid_ridge.best_score_)
```

```
0.9121212051696579
```

In [50]:
```
best_ridge = grid_ridge.best_estimator_

ridge_pred =best_ridge.predict(X_test)
```

In [51]:
```
print("MAE :", mean_absolute_error(y_test,ridge_pred))

print("MSE :", mean_squared_error(y_test,ridge_pred))

print("RMSE :", np.sqrt(mean_squared_error(y_test,ridge_pred)))

print("R2 Score :", r2_score(y_test,ridge_pred))
```

```
MAE : 79277.25338222191
MSE : 9992756317.455666
RMSE : 99963.77502603464
R2 Score : 0.914832076303878
```

Lasso Regression

In [52]:
```
from sklearn.linear_model import Lasso
param_grid = {'alpha' : [0.001,0.01,0.1,1,10]}
```

In [53]:
```
grid_lasso = GridSearchCV(Lasso(max_iter=5000), param_grid, cv=5,scoring='r2')

grid_lasso.fit(X_train,y_train)
```

Out[53]:
```
GridSearchCV(cv=5, estimator=Lasso(max_iter=5000),
             param_grid={'alpha': [0.001, 0.01, 0.1, 1, 10]}, scoring='r2')
```
In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.

In [55]:
```
print(grid_lasso.best_params_)
```

```
{'alpha': 10}
```

In [56]:
```
lasso_pred = grid_lasso.predict(X_test)

print(r2_score(y_test,lasso_pred))
```

```
0.9148280193164746
```

In [ ]: